

# Fonctions Detaillees des Agents 38 a 42

Le vrai code de chaque agent, fonction par fonction : le code exact, puis pourquoi il est ecrit ainsi.

---

## 38. SiteMap-Firecrawl

*38-SiteMap-Firecrawl/map\_agent.py*

### **explore\_site(url, search, limit) -- RECUPERATION**

```
def explore_site(url: str, search: str | None = None, limit: int = MAP_LIMIT) -> list[Link]:
    """Decouvre les URLs d'un site via Firecrawl map (rapide, sans scraping de contenu)."""
    result = firecrawl.map(url, search=search, limit=limit)
    links = []
    for item in result.links or []:
        if not item.url:
            continue
        links.append(Link(url=item.url, title=item.title or item.url))
    return links
```

#### **POURQUOI**

C'est la fonction de recuperation : elle isole l'appel a firecrawl.map() du reste du programme. Le "if not item.url: continue" gere le cas ou Firecrawl renvoie un lien incomplet -- on l'ignore silencieusement plutot que de planter toute la cartographie pour un seul lien mal forme. Cette fonction ne sait rien du LLM ni de l'interface : elle fait UNE chose, recuperer une liste de liens propres.

### **map\_site(url, search, limit) -- ORCHESTRATION**

```
def map_site(url: str, search: str | None = None, limit: int = MAP_LIMIT) -> tuple[list[Link], list[Category]]:
    """Decouvre puis categorise les URLs d'un site. Retourne (liens bruts, categories)."""
    links = explore_site(url, search=search, limit=limit)
    categories = categorize(links)
    return links, categories
```

#### **POURQUOI**

C'est la fonction que app.py appelle -- le seul point d'entree. Elle connait l'ordre exact (explorer PUIS categoriser), mais ne sait pas COMMENT chacune de ces deux etapes fonctionne en detail. Si demain on change categorize() pour utiliser un autre LLM, map\_site() n'a pas besoin de changer une seule ligne.

---

## 39. SiteCrawl-Firecrawl

*39-SiteCrawl-Firecrawl/crawl\_agent.py*

## **crawl\_site(url, limit) -- RECUPERATION**

```
def crawl_site(url: str, limit: int = CRAWL_LIMIT) -> list[Page]:
    """Explore un site a partir de `url` (en suivant les liens) et scrape chaque page visitee."""
    job = firecrawl.crawl(url, limit=limit, scrape_options=ScrapeOptions(formats=["markdown"]))
    pages = []
    for doc in job.data or []:
        markdown = (getattr(doc, "markdown", None) or "").strip()
        metadata = getattr(doc, "metadata", None)
        page_url = getattr(metadata, "url", None) if metadata else None
        title = (getattr(metadata, "title", None) if metadata else None) or page_url
        if not page_url or not markdown:
            continue
        pages.append(Page(title=title, url=page_url, content=markdown[:MAX_CHARS_PER_PAGE]))
    return pages
```

### **POURQUOI**

Plus defensive que celle de l'agent 38 : `firecrawl.crawl()` visite plusieurs pages, et CHACUNE peut echouer independamment (page bloquee, JS-only...). Les `getattr(..., None)` partout evitent un crash si une page renvoie un objet incomplet. Le `[:MAX_CHARS_PER_PAGE]` tronque chaque page pour que le total envoye au LLM plus tard reste sous la limite de tokens.

## **digest(url, limit) -- ORCHESTRATION**

```
def digest(url: str, limit: int = CRAWL_LIMIT) -> tuple[str, list[Page]]:
    """Crawl le site puis demande au LLM une fiche de synthese sourcee."""
    pages = crawl_site(url, limit=limit)
    if not pages:
        return "Aucune page exploitable n'a ete trouvee..", []
    pages_block = "\n\n".join(
        f"[{i}] {p.title} ({p.url})\n{p.content}" for i, p in enumerate(pages, 1)
    )
    prompt = f"Site explore : {url}\n\nPages : \n\n{pages_block}"
    report = digest_agent.run(prompt)
    return report.content, pages
```

### **POURQUOI**

Le "if not pages: return" avant d'appeler le LLM est important : sans lui, on paierait un appel LLM pour lui demander de resumer... rien. La numerotation `[i]` construite ici correspond exactement a la numerotation `[1][2]` que l'agent utilise dans ses citations -- c'est ce lien qui rend le rapport final veritablement source et pas juste plausible.

---

## **40. PageAnalyzer-Firecrawl**

*40-PageAnalyzer-Firecrawl/scrape\_agent.py*

## **scrape\_page(url) -- RECUPERATION**

```
def scrape_page(url: str) -> Optional[str]:
    """Scrape une page via Firecrawl et retourne son contenu markdown (None si echec)."""
    result = firecrawl.scrape(url, formats=["markdown"])
    markdown = getattr(result, "markdown", None)
    if not markdown or not markdown.strip():
        return None
    return markdown.strip()
```

### **POURQUOI**

La plus simple des 5 fonctions de recuperation, parce qu'elle ne traite qu'UNE page (pas de boucle). Le type de retour `Optional[str]` (vu plus tot dans nos echanges) documente explicitement que None est un resultat possible -- ce qui force `analyze_page()` a le verifier avant de continuer.

## analyze\_page(url, question) -- ORCHESTRATION

```
def analyze_page(url: str, question: Optional[str] = None) -> tuple[Optional[str], Optional[str]]:
    """Scrape `url` puis repond a `question` (ou resume la page si absente)."""
    content = scrape_page(url)
    if content is None:
        return None, "Impossible d'extraire le contenu de cette page..."
    truncated = content[:MAX_CONTENT_CHARS]
    if question and question.strip():
        prompt = f"Question : {question.strip()}\n\nContenu de la page ({url}) :\n\n{truncated}"
    else:
        prompt = f"Resume cette page ({url}) :\n\n{truncated}"
    answer = page_agent.run(prompt)
    return answer.content, None
```

### POURQUOI

Cette orchestration a une branche supplementaire que les autres n'ont pas : elle construit un PROMPT DIFFERENT selon que l'utilisateur a pose une question ou non. C'est elle qui decide du "mode" (question-reponse vs resume), pas l'agent LLM lui-meme -- l'agent recoit juste un prompt deja pret a l'emploi.

---

## 41. DocumentCompare-MistralOCR

*41-DocumentCompare-MistralOCR/compare\_agent.py*

### ocr\_pdf(file\_path) -- RECUPERATION

```
def ocr_pdf(file_path: str) -> str:
    """Extrait le contenu markdown d'un PDF via l'OCR Mistral."""
    uploaded = mistral_client.files.upload(
        file={"file_name": os.path.basename(file_path), "content": open(file_path, "rb")},
        purpose="ocr",
    )
    signed_url = mistral_client.files.get_signed_url(file_id=uploaded.id)
    ocr_response = mistral_client.ocr.process(
        model="mistral-ocr-latest",
        document={"type": "document_url", "document_url": signed_url.url},
        include_image_base64=False,
    )
    return "\n".join(page.markdown for page in ocr_response.pages)
```

### POURQUOI

Recuperation en 3 etapes obligatoires cote API Mistral (upload, URL signee, OCR) -- mais du point de vue du reste du programme, tout ca reste cache derriere UNE fonction qui prend un chemin de fichier et renvoie du texte. C'est exactement le role d'une fonction de recuperation : cacher la complexite d'un service externe derriere une interface simple.

### compare\_pdfs(path\_a, path\_b) -- ORCHESTRATION

```
def compare_pdfs(path_a: str, path_b: str) -> tuple[ComparisonResult, str, str]:
    """OCR les deux PDFs puis les compare. Retourne (resultat, texte_a, texte_b)."""
    text_a = ocr_pdf(path_a)
    text_b = ocr_pdf(path_b)
    result = compare_documents(text_a, text_b)
    return result, text_a, text_b
```

### POURQUOI

La plus courte des 5 orchestrations, mais elle illustre bien le principe : 3 lignes qui enchainent 2 appels de recuperation (un par fichier) puis 1 appel de decision (compare\_documents, qui elle-meme appelle le LLM). Aucune des 3 fonctions appelees ne sait qu'elle fait partie d'une comparaison -- seule compare\_pdfs() connait le "scenario" complet.

---

## 42. ResearchTeam-Agno

*42-ResearchTeam-Agno/research\_team.py*

**firecrawl\_search(query) -- RECUPERATION (mais utilisee comme outil, pas appelee directement)**

```
def firecrawl_search(query: str) -> str:
    """Recherche sur le web via Firecrawl et renvoie le contenu (markdown) des
    meilleurs resultats, numerotes [1], [2]... pour permettre des citations."""
    result = firecrawl.search(query, limit=SEARCH_LIMIT, scrape_options=ScrapeOptions(formats=["markdown"]))
    blocks = []
    for i, item in enumerate(result.web or [], 1):
        ...
        blocks.append(f"[{i}] {title} ({url})\n{markdown[:MAX_CHARS_PER_SOURCE]}")
    return "\n\n".join(blocks) if blocks else "Aucun resultat exploitable..."
```

#### POURQUOI

Difference cle avec les 4 autres agents : cette fonction n'est PAS appelee directement par une fonction d'orchestration ecrite a la main. Elle est passee en tant qu'OUTIL a l'agent Chercheur (tools=[firecrawl\_search]) -- c'est le LLM lui-meme qui decide quand et combien de fois l'appeler. Le docstring n'est pas juste de la documentation ici : c'est ce texte que le LLM lit pour savoir QUAND utiliser cet outil.

**run\_research(query) -- ORCHESTRATION**

```
def run_research(query: str) -> tuple[Optional[str], Optional[str]]:
    """Lance l'equipe sur `query`. Retourne (rapport, erreur)."""
    try:
        result = research_team.run(query)
    except Exception as e:
        return None, f"Erreur pendant l'execution de l'equipe : {e}"
    content = result.content
    if not isinstance(content, str) or not content.strip():
        return None, "L'equipe n'a pas produit de rapport exploitable..."
    if content.strip().startswith('{ "error" }'):
        return None, "Le fournisseur du modele a temporairement refuse..."
    return content, None
```

#### POURQUOI

L'orchestration la plus defensivaive des 5, ecrite APRES avoir observe un vrai echec en test (Groq a renvoye une erreur JSON brute comme si c'etait un rapport valide). Elle ne se contente pas d'appeler l'equipe : elle verifie aussi que ce qu'elle a recu ressemble a un vrai rapport avant de le transmettre a l'interface -- une orchestration protege aussi contre les echecs SILENCIEUX d'un service externe, pas seulement contre les exceptions.

---